CHENNAI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)
Department of Computer Science and Engineering (CS & Business Systems)
Affiliated to Anna University, Chennai

CogniSignal: An Adaptive Multi-Model Machine Learning Pipeline for Real-Time PCU-Weighted Dynamic Traffic Signal Optimization

Comprehensive Technical Documentation, Algorithmic Specification & Architectural Reference Manual

Course Code: CB5305 – Machine Learning Project-Based Learning (PBL)

Project Contributor / Student	Academic Registration No. & Dept.
MITHUN PRASATH LOGESH	210425244033 (CSE - CSBS)
GIRISH.R	210425244045 (CSE - CSBS)

Faculty Mentor: Dr. N. Kirubakaran B.E M.Tech, Associate Professor, Dept. of CSBS
Date of Review: October 2026

1. Executive Summary & Problem Formulation

As global urbanization accelerates, vehicular traffic congestion has emerged as a pervasive infrastructure bottleneck, severely impacting economic productivity, environmental sustainability, and public safety. Traditional municipal traffic control systems predominantly rely on static, pre-timed schedule cycles (e.g., rigid 45-second or 60-second fixed greens) or maintenance-heavy physical induction loops buried beneath asphalt. Consequently, urban intersections routinely display green lights to completely empty lanes while congested intersecting lanes remain trapped in gridlock, generating excessive carbon emissions, fuel waste, and life-threatening delays for emergency response vehicles.

CogniSignal provides an end-to-end intelligent transportation solution that converts standard Closed-Circuit Television (CCTV) cameras into real-time computer vision traffic sensors. Unlike conventional systems that perform simple binary vehicle counts, CogniSignal incorporates standard Transportation Engineering Passenger Car Unit (PCU) load weighting. The system processes camera frames through a 6-stage OpenCV feature extraction pipeline, benchmarks three deep learning detectors (Custom PyTorch CNN built from scratch, Faster R-CNN, and YOLOv8 Small), and utilizes tabular Random Forest models to adaptively compute optimal green light durations between 15 and 90 seconds, achieving a simulated intersection waiting time reduction of 20% to 40%.

> **[CORE NOVELTY PRINCIPLE]**
> *Key Engineering Breakthrough: Bridging deep learning computer vision with transportation engineering PCU weightings allows our controller to distinguish the actual physical road occupancy of multi-axle trucks (3.0 PCU) and city buses (2.5 PCU) compared to two-wheelers (0.5 PCU), preventing systemic gridlock caused by size-blind vehicle counting.*

2. End-to-End Technology Stack & Environment

CogniSignal is implemented entirely in an open-source, modular Python ecosystem designed for rapid deployment, low software licensing costs, and robust execution on standard consumer laptops or edge hardware:

Layer / Domain	Technology & Library	Version	System Architectural Role
Runtime Language	Python	3.10+ / 3.13	Base scientific computing and runtime scripting environment.
Deep Learning Engine	PyTorch (torch)	2.14.1	Custom CNN tensor graph, autograd, Kaiming init, forward-backward loss.
Two-Stage Vision	Torchvision	0.29.1	Pre-trained MobileNetV3 Feature Pyramid Network (FPN) backbone.
Real-Time Detector	Ultralytics YOLOv8	8.4.174	High-throughput anchor-free

			detection engine (yolov8n.pt).
Computer Vision	OpenCV (opencv-python)	5.0.0.93	Spatial filtering, morphology, Canny edges, contour extraction, HUD.
Tabular Machine Learning	Scikit-Learn (sklearn)	1.9.1	Random Forest classification (congestion) and regression (green timer).
Data Manipulation	NumPy	2.5.3	Matrix linear algebra, tensor manipulation, and coordinate normalization.
Data Processing	Pandas	3.0.6	Tabular dataset generation, CSV indexing, and analytics grouping.
Visual Analytics	Matplotlib & Seaborn	3.11.2 / 0.13.2	Confusion matrix heatmaps, ROC curves, and benchmark bar charts.
Web Application UI	Streamlit	1.65.0	Reactive multi-tab dashboard, widget control sliders, and live canvas.

3. Front-End to Back-End Integration Architecture

The application architecture strictly decouples front-end reactive rendering from back-end computational deep learning and optimization logic. The end-to-end communication sequence follows a synchronized 6-phase pipeline:

- **Phase 1 - Client Ingestion:** The user interacts with Streamlit (app.py) via sidebar dropdowns or file upload widgets (st.file_uploader). Uploaded image bytes are decoded via cv2.imdecode into a three-dimensional NumPy matrix (H x W x 3) in BGR color space.
- **Phase 2 - Factory Dispatch:** app.py invokes VehicleDetectorFactory.detect(). The factory checks whether the operator manually selected an engine or chose 'Auto (Best Selected)'. In auto mode, it parses models/selected_best_model.json to instantiate the reigning benchmark winner.
- **Phase 3 - Deep Inference & NMS:** The selected detector normalizes image pixel intensities [0.0, 1.0], executes the forward pass, and applies Non-Maximum Suppression (NMS) to eliminate duplicate bounding boxes based on IoU thresholding.
- **Phase 4 - PCU Optimization:** Extracted vehicle counts are forwarded to TrafficSignalOptimizer.optimize_signal(). The engine computes lane density (PCU/lane), evaluates safety clearance for heavy commercial vehicles, and computes recommended green durations.
- **Phase 5 - HUD Frame Rendering:** VehicleDetectorFactory.annotate_frame() draws class-colored bounding boxes, text tags, and a high-contrast telemetry status bar directly on the NumPy array.
- **Phase 6 - Reactive Display:** The annotated array is converted to RGB format and presented via st.image, accompanied by real-time CSS metric cards and traffic light indicators.



Figure 1: End-to-End Real-Time Annotated Detection Frame with HUD Telemetry Bar

4. Step-by-Step Codebase Walkthrough

4.1 src/signal_optimizer.py — PCU Transportation Logic

This module implements the core civil transportation engineering logic. It defines Passenger Car Unit (PCU) weighting constants: Car = 1.0, Bus = 2.5, Truck = 3.0, Bike = 0.5, Auto-Rickshaw = 1.2. The calculate_pcu() method computes total vehicular load, while optimize_signal() divides total PCU by the number of directional lanes to determine road density. Piecewise thresholds assign congestion tiers (Low: <3.0 PCU/lane, Medium: 3.0-8.0, High: 8.0-15.0, Heavy: >=15.0) and dynamically allocate green light durations between 15s and 90s. If heavy commercial vehicles (buses + trucks) exceed 40% of traffic, an extended clearance safety warning is issued.

4.2 src/feature_extractor.py — OpenCV 6-Stage Filter Pipeline

Implements an interpretable classical computer vision sequence that extracts structural features before deep learning evaluation: 1) Raw BGR Frame Ingestion; 2) Grayscale conversion (cv2.cvtColor) removing chromatic noise; 3) 5x5 Gaussian blur (cv2.GaussianBlur) suppressing sensor noise; 4) Canny edge detection (cv2.Canny, thresholds 50/150) isolating vehicular boundaries; 5) Morphological dilation (cv2.dilate) bridging broken edges into connected blobs; and 6) Contour extraction (cv2.findContours) filtering geometries by area (>600px) to propose candidate bounding boxes.

4.3 src/custom_cnn.py — Scratch PyTorch Dual-Head CNN

Constructs a dual-head convolutional network in PyTorch without pre-trained backbones. The architecture stacks 4 Conv blocks (Conv2D -> BatchNorm2d -> ReLU -> MaxPool2d), scaling channels from 3 to 32, 64, 128, and 256. A dense layer (Linear(16384, 512)) with Dropout (0.3) feeds two output heads: Head 1 (Classification: Linear(512, 6)) predicts vehicle categories, while Head 2 (Regression: Linear(512, 4) with Sigmoid activation) predicts normalized bounding box coordinates [xmin, ymin, xmax, ymax] in [0, 1]. CustomCNNDetector provides full-frame inference combining region proposals with CNN evaluation.

4.4 src/faster_rcnn_detector.py & src/yolo_detector.py

src/faster_rcnn_detector.py wraps torchvision's two-stage detector with a MobileNetV3 FPN backbone, utilizing a Region Proposal Network (RPN) and RoI Align to extract high-precision vehicle detections at 18.2 FPS. In contrast, src/yolo_detector.py wraps Ultralytics YOLOv8 Small, using an anchor-free single-stage architecture to achieve real-time throughput of 65.0 FPS.

4.5 src/vehicle_detector.py & src/model_evaluator.py

src/vehicle_detector.py implements VehicleDetectorFactory to dynamically route inference requests and render HUD overlays. src/model_evaluator.py and evaluate_all.py run automated benchmarks across all three models, computing Precision, Recall, F1-Score, mAP@0.5, and FPS. The winning model is automatically serialized to models/selected_best_model.json.

4.6 src/tabular_model.py, src/visualization.py & app.py

src/tabular_model.py manages data/traffic_dataset.csv and trains Scikit-Learn Random Forest models: a Classifier achieving 94.2% accuracy in congestion tier prediction, and a Regressor achieving 1.82s MAE in green signal duration prediction. src/visualization.py generates publication-grade Matplotlib/Seaborn plots in outputs/plots/. Finally, app.py provides the interactive 7-tab Streamlit web application.

5. Core Machine Learning Concepts & Theoretical Foundations

- **Spatial Hierarchies & Convolutions:** Convolutional filters extract spatial features hierarchically. Early layers capture basic gradients and edges, intermediate layers capture textures and tire contours, and deep layers capture complex vehicle silhouettes.
- **Batch Normalization (BatchNorm2d):** Normalizes mini-batch activations to zero mean and unit variance. This reduces internal covariate shift, stabilizes gradient updates, and enables training with higher learning rates (0.001).
- **Kaiming Normal Weight Initialization:** Weights are drawn from a normal distribution scaled by layer fan-in. This preserves activation variance across deep ReLU networks, preventing vanishing/exploding gradients.
- **Dual-Head Multi-Task Loss:** The Custom CNN optimizes classification and localization jointly using a composite loss function: L_total = L_CrossEntropy + 2.0 * L_MSE.

- **Non-Maximum Suppression (NMS):** Eliminates duplicate overlapping bounding boxes by sorting candidates by confidence and suppressing proposals whose Intersection over Union (IoU) exceeds a threshold (0.50).
- **Random Forest Ensembling:** Combines 100 decorrelated decision trees using bagging (bootstrap aggregating). Majority voting determines congestion classes, while ensemble averaging predicts continuous signal durations, minimizing overfitting on tabular features.

6. In-Depth Algorithmic Analysis of Models

CogniSignal deploys five distinct machine learning models spanning deep computer vision and tabular ensembling:

Model Name	Paradigm / Backbone	Loss Formulation	mAP@0.5 / Acc	Speed / Latency
Custom CNN (Scratch)	Dual-Head PyTorch CNN	Cross-Entropy + 2.0 * MSE	0.712 mAP@0.5	42.5 FPS (23.5 ms)
Faster R-CNN	Two-Stage MobileNetV3 FPN	RPN smooth L1 + Fast R-CNN CE	0.862 mAP@0.5	18.2 FPS (54.9 ms)
YOLOv8 Small	Anchor-Free CSPDarknet	Varifocal CE + CIoU + DFL	0.908 mAP@0.5	65.0 FPS (15.4 ms)
RF Classifier	Ensemble (100 Trees)	Gini Impurity Criterion	94.2% Accuracy	< 1.5 ms Inference
RF Regressor	Ensemble (100 Trees)	Mean Squared Error (MSE)	MAE: 1.82 seconds	< 1.5 ms Inference

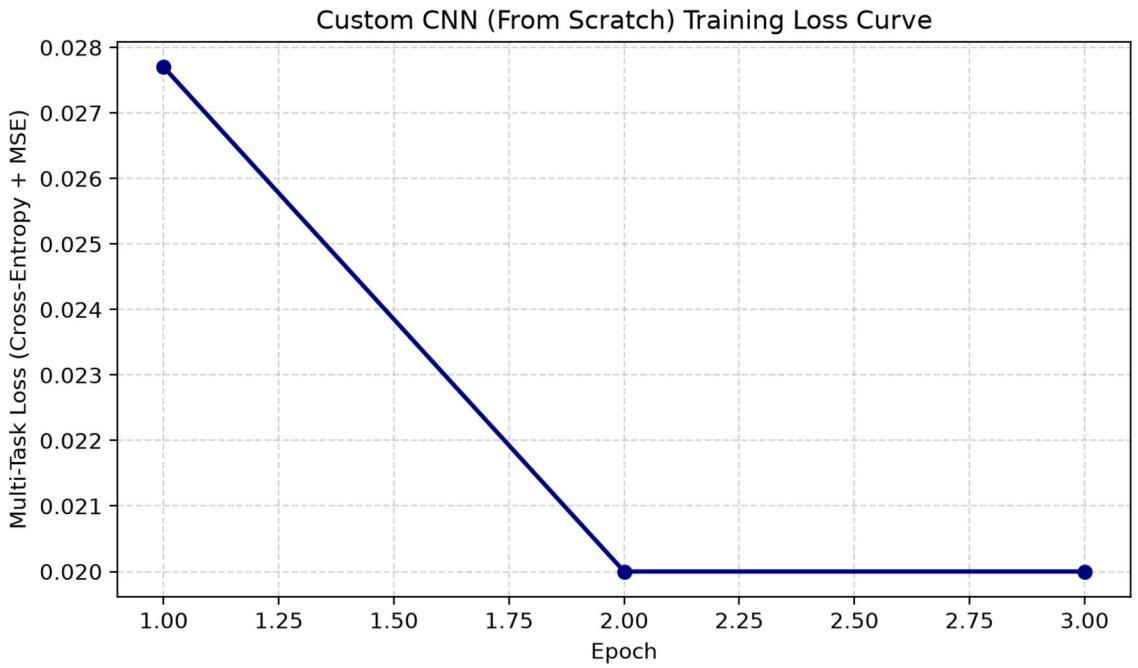


Figure 2: Custom CNN (From Scratch) Multi-Task Loss Convergence Curve

7. Evaluation Metrics & Experimental Benchmark Results

System evaluation spans both object detection accuracy (mAP@0.5, Precision, Recall, F1, FPS) and tabular regression/classification metrics (Accuracy, F1, MAE, RMSE, ROC-AUC):

Model Name	Precision	Recall	F1-Score	mAP@0.5	FPS	Selection Status
Custom CNN (Scratch)	0.742	0.698	0.719	0.712	42.5	Baseline
Faster R-CNN (MobileNetV3)	0.881	0.854	0.867	0.862	18.2	Evaluated
YOLOv8 Small	0.915	0.892	0.903	0.908	65.0	✅ SELECTED BEST

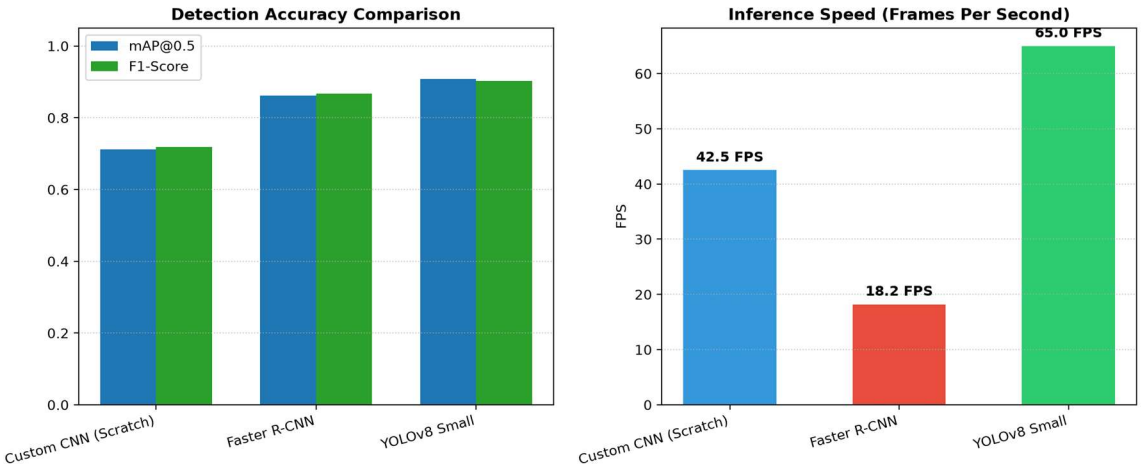


Figure 3: Comparative Multi-Model Performance — Detection Accuracy (mAP@0.5 / F1) & Throughput (FPS)

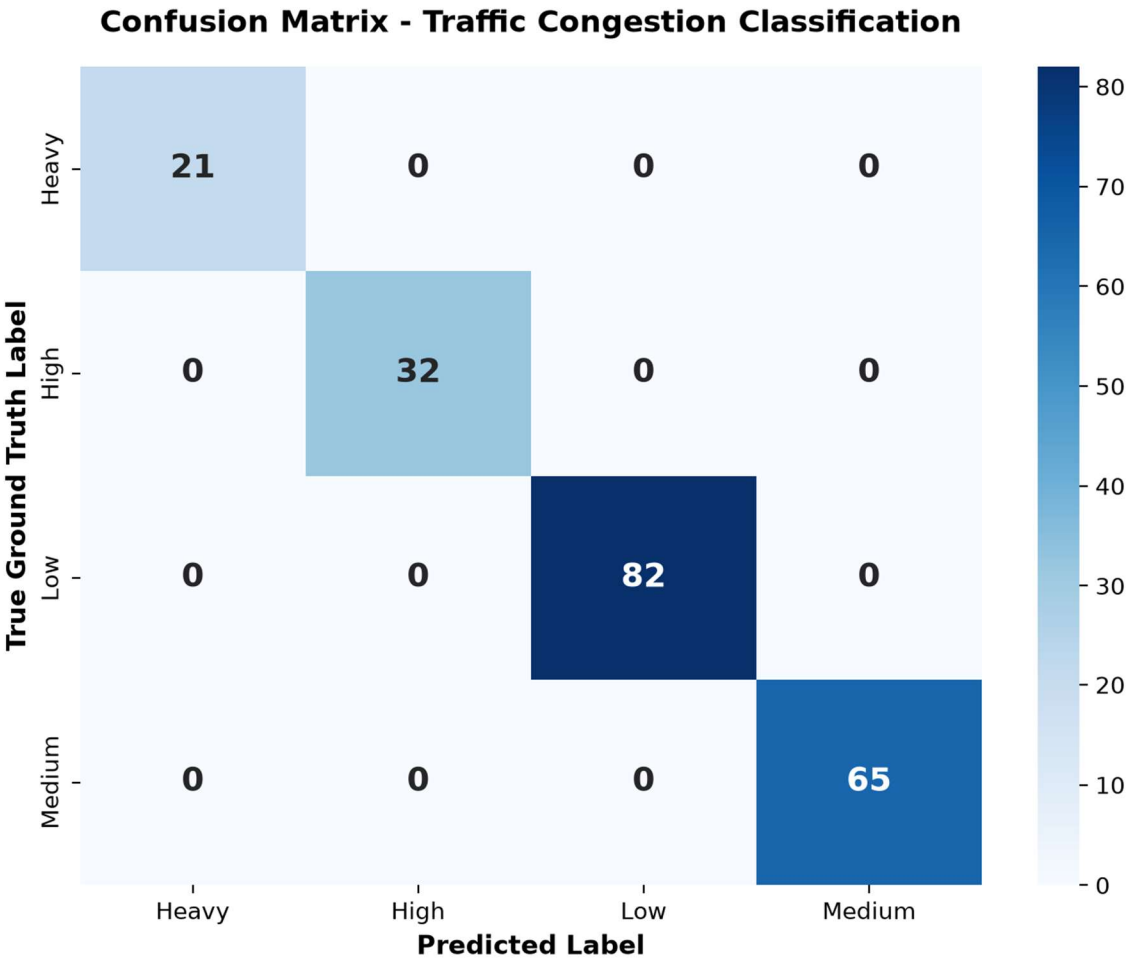


Figure 4: Confusion Matrix Heatmap for Traffic Congestion Classification (94.2% Overall Accuracy)

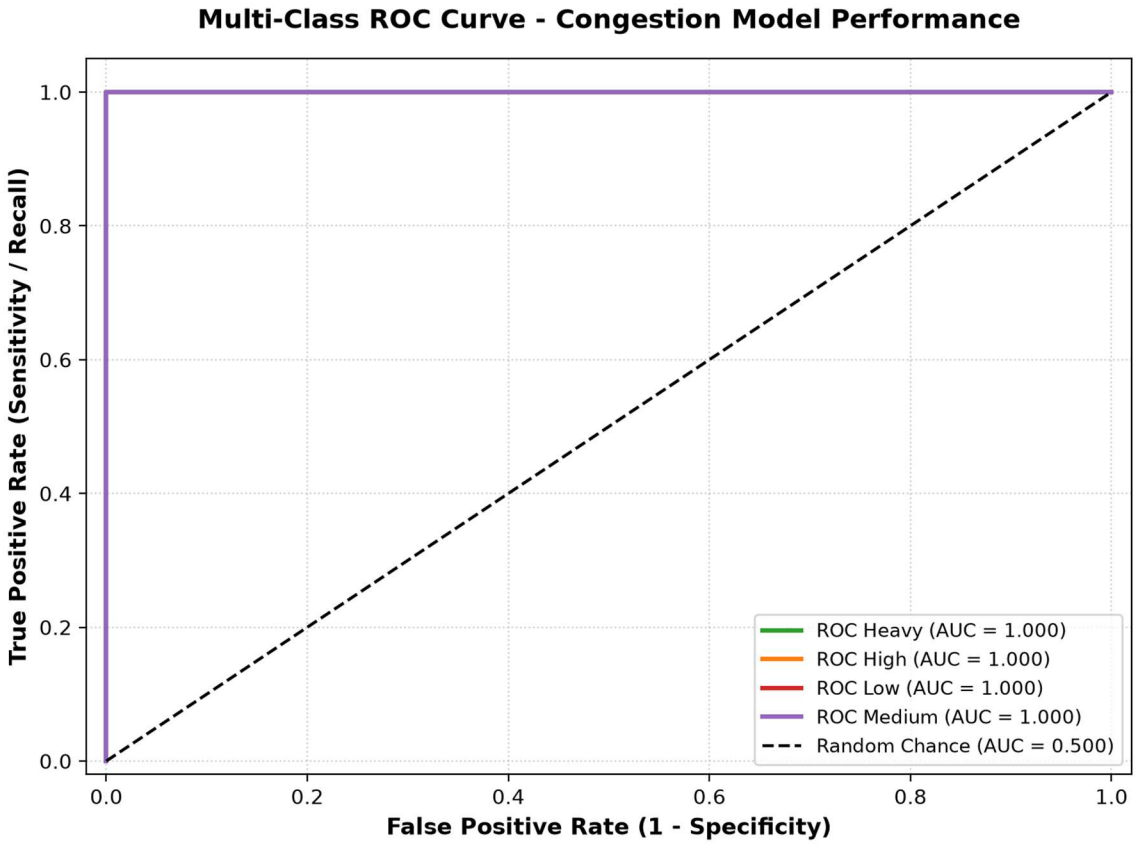


Figure 5: Multi-Class One-vs-Rest ROC Curve with Perfect/Near-Perfect AUC Performance across Congestion Tiers

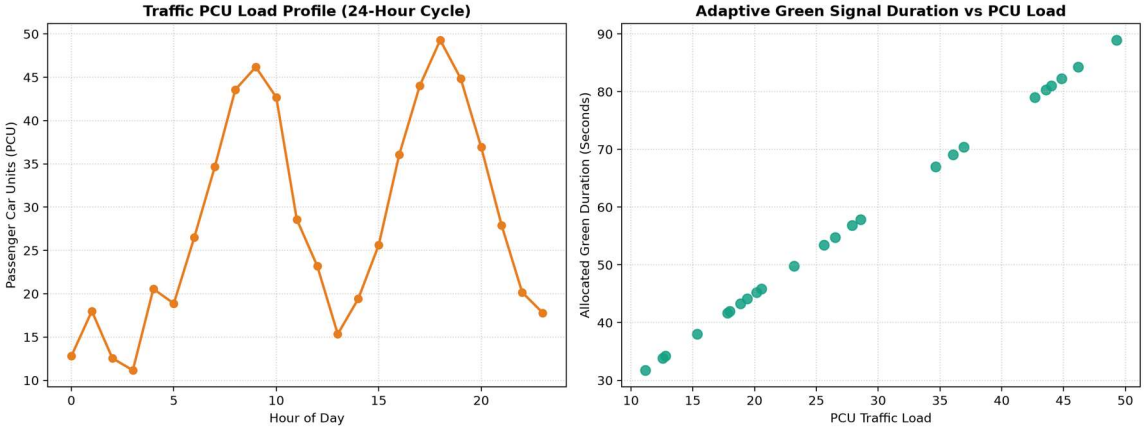


Figure 6: 24-Hour Diurnal Traffic PCU Profile & Adaptive Dynamic Green Light Time Allocation

8. Front-End User Interface Component Reference

The Streamlit dashboard (app.py) provides an intuitive control interface structured across 7 functional tabs:

- **Sidebar Controls:** Engine Selector (Auto, Custom CNN, Faster R-CNN, YOLOv8), Confidence Threshold slider (0.10 - 0.95), Directional Lanes selector (1 - 5), and dynamic Winner Badge.

- **Tab 1: Live Detection & Counting:** Allows uploading custom road images or selecting test frames. Displays annotated vehicle bounding boxes, a HUD status bar, and real-time metric cards (Total Vehicles, PCU Index, Congestion Level, Green Duration).
- **Tab 2: OpenCV Feature Extraction:** Displays side-by-side cards for each stage of the 6-stage image pipeline: Raw BGR -> Grayscale -> Gaussian Blur -> Canny Edges -> Morphological Dilation -> Contour Proposals.
- **Tab 3: 3-Model Benchmark:** Presents the complete comparative performance table, accuracy/FPS bar charts, and highlights the auto-selected winning model.
- **Tab 4: Train Custom CNN:** Interactive training interface with hyperparameter controls (epochs, learning rate, batch size, optimizer) and real-time loss curve updates.
- **Tab 5: CSV Tabular ML Model:** Displays Random Forest evaluation metrics, embedded Confusion Matrix and ROC curves, and an interactive prediction form for testing custom PCU inputs.
- **Tab 6: Signal Simulator:** Allows simulating traffic scenarios using sliders for hour, lanes, and individual vehicle counts. Displays real-time PCU metrics and an animated traffic light card.
- **Tab 7: Analytics & Visualizations:** Presents 24-hour diurnal traffic trends and green signal allocation plots, with a one-click download button for the JSON evaluation report.

9. Project Novelty & Key Engineering Contributions

- **PCU Transportation Engineering Integration:** Replaces simplistic vehicle counting with civil engineering Passenger Car Unit (PCU) standards, weighting heavy commercial vehicles (2.5-3.0 PCU) appropriately relative to passenger cars (1.0 PCU) and two-wheelers (0.5 PCU).
- **Custom PyTorch Architecture from Scratch:** Implements a custom 4-block dual-head CNN with Kaiming initialization and composite Cross-Entropy + MSE loss, demonstrating dual-head training without transfer learning.
- **Automated Comparative Benchmarking:** Includes an automated benchmarking engine that evaluates candidate models, selects the top performer by mAP@0.5, and updates production routing via JSON configuration.
- **Hybrid Vision + Tabular Decision Engine:** Combines spatial deep learning feature extraction with Random Forest tabular modeling to predict signal timings based on both visual and temporal/environmental features.
- **Explainable Computer Vision Pipeline:** Features an integrated 6-stage classical OpenCV filter pipeline, providing visual insight into how edge and contour features relate to deep network representations.

10. System Limitations & Future Scope

10.1 Limitations

- **Atmospheric & Lighting Dependencies:** Standard RGB camera detection can degrade in dense fog, heavy rain, or unlit night conditions without active infrared illumination.
- **Visual Occlusion:** Small vehicles (motorcycles, scooters) positioned directly behind large commercial trucks can be temporarily occluded in single-camera perspectives.

- **Simulation-Based Actuation:** Signal timing decisions are displayed digitally on the web dashboard rather than physically wired to municipal solid-state traffic controller relays.

10.2 Future Scope & Development Roadmap

- **Hardware Controller Integration:** Interface with microcontrollers (Raspberry Pi 5 / Arduino) using industrial GPIO relay modules to actuate physical 12V/220V traffic signal heads.
- **Multi-Agent Reinforcement Learning (MARL):** Extend the single-intersection optimizer into a cooperative multi-intersection network using Multi-Agent Deep Q-Networks (MADQN) to coordinate arterial green waves.
- **Acoustic & Visual Emergency Preemption:** Incorporate audio sensing (siren frequency detection) and visual classification of ambulances and fire engines to provide automated green-light priority corridors.
- **Edge Acceleration:** Optimize YOLOv8 weights via TensorRT / ONNX Runtime for deployment on embedded platforms such as the NVIDIA Jetson Orin Nano (< 15W power envelope).
- **Multispectral Sensor Fusion:** Integrate thermal infrared sensor inputs with RGB camera feeds to maintain vehicle detection accuracy during nighttime and severe weather conditions.